

INT 221 - MVC PROGRAMMING

Unit 2

Laravel Request Lifecycle

Route

Controller

View

**Seems but not true
General Answer
But it is not True**

Lifecycle

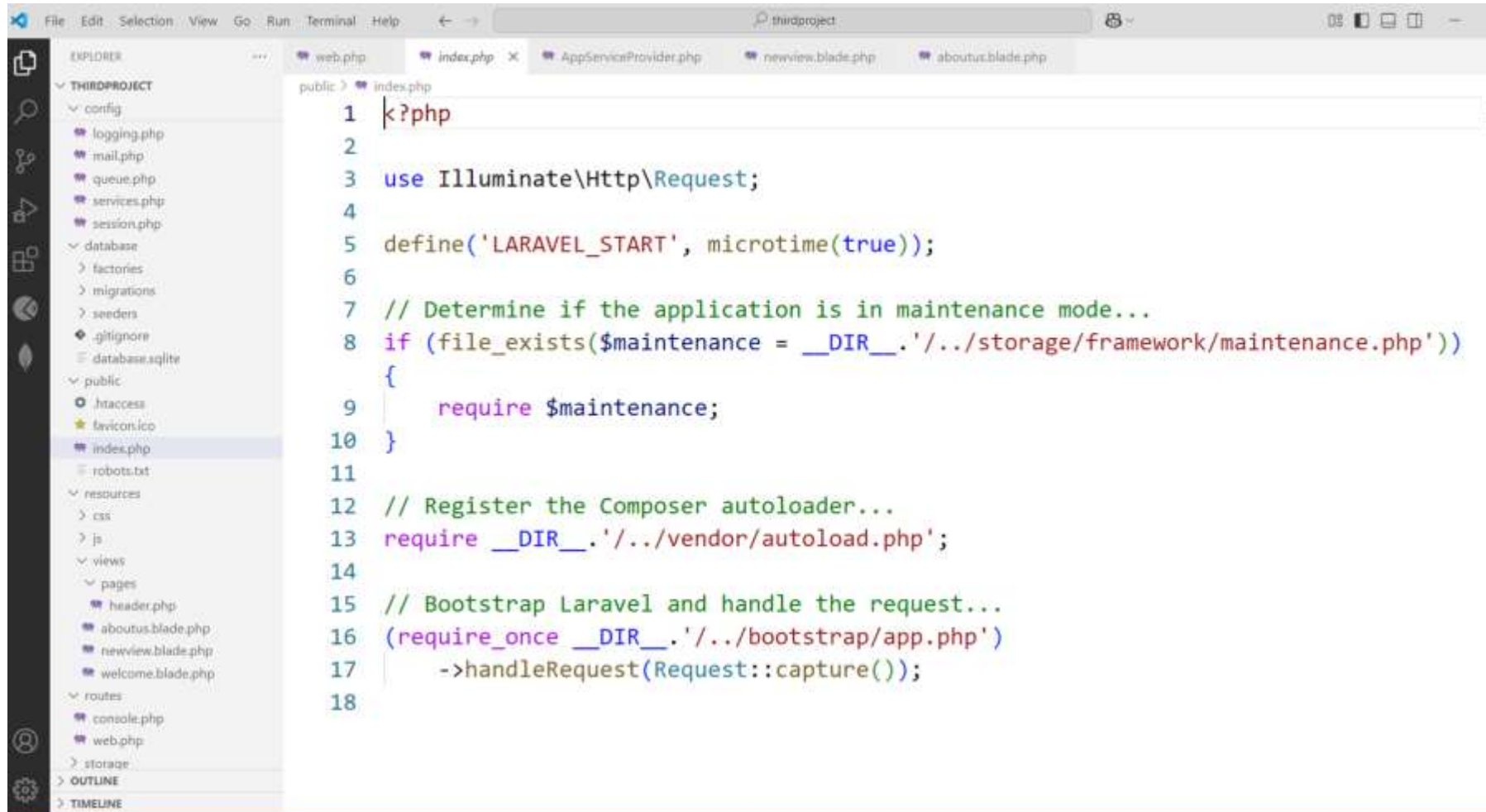
a **sequence of events and actions that occur during the execution of a request.** Each request goes through several stages, and Laravel provides various mechanisms to perform actions at different points in this lifecycle

Index.php – The **ENTRY** point

All **requests are directed to this file by your web server** (Apache / Nginx) configuration. The index.php file doesn't contain much code. Rather, it is a **starting point for loading the rest of the framework**.

The **index.php** file loads the Composer-generated **autoloader definition** and then **retrieves an instance of the Laravel application** from bootstrap/app.php.

Structure of index.php file



The screenshot shows a code editor with the following content:

```
1 k?php
2
3 use Illuminate\Http\Request;
4
5 define('LARAVEL_START', microtime(true));
6
7 // Determine if the application is in maintenance mode...
8 if (file_exists($maintenance = __DIR__.'/../storage/framework/maintenance.php'))
9 {
10     require $maintenance;
11 }
12
13 // Register the Composer autoloader...
14 require __DIR__.'/../vendor/autoload.php';
15
16 // Bootstrap Laravel and handle the request...
17 (require_once __DIR__.'/../bootstrap/app.php')
18     ->handleRequest(Request::capture());
```

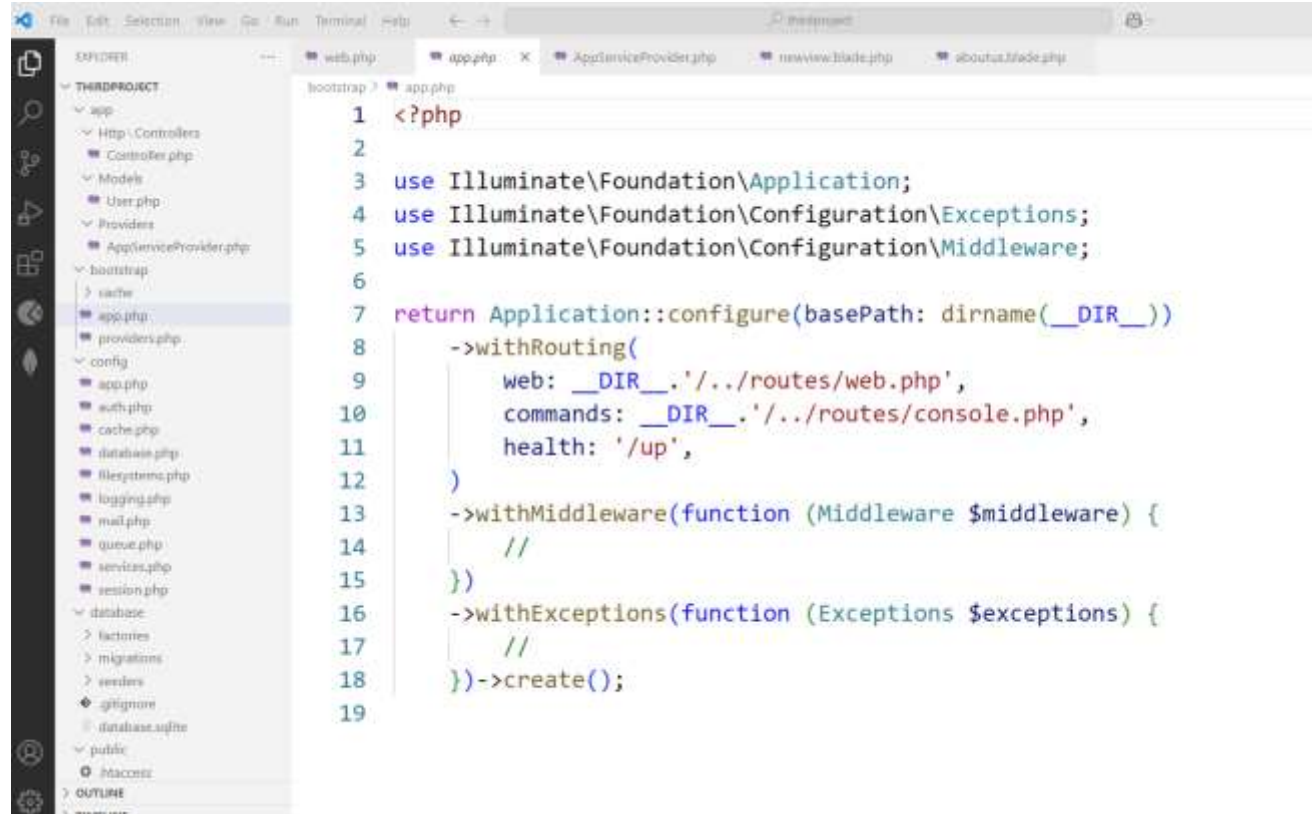
The left sidebar shows the file explorer with the following structure:

- THIRDPROJECT
 - config
 - logging.php
 - mail.php
 - queue.php
 - services.php
 - session.php
 - database
 - factories
 - migrations
 - seeders
 - .gitignore
 - database.sqlite
 - public
 - .htaccess
 - favicon.ico
 - index.php
 - robots.txt
 - resources
 - css
 - js
 - views
 - pages
 - header.php
 - aboutus.blade.php
 - newview.blade.php
 - welcome.blade.php
 - routes
 - console.php
 - web.php
 - storage
- OUTLINE
- TIMELINE

Composer Autoloader

An autoloader in Laravel is like a **smart library organizer for your code**. It **automatically finds and loads the right files when you use different parts of your Laravel project**. This way, you don't have to search for and include files manually, making your coding life easier and more organized.

Bootstrap -> app.php (Instances are made here)



```
1 <?php
2
3 use Illuminate\Foundation\Application;
4 use Illuminate\Foundation\Configuration\Exceptions;
5 use Illuminate\Foundation\Configuration\Middleware;
6
7 return Application::configure(basePath: dirname(__DIR__))
8     ->withRouting(
9         web: __DIR__.'/../routes/web.php',
10        commands: __DIR__.'/../routes/console.php',
11        health: '/up',
12    )
13    ->withMiddleware(function (Middleware $middleware) {
14        //
15    })
16    ->withExceptions(function (Exceptions $exceptions) {
17        //
18    })->create();
19
```

The bootstrap/app.php file sets up the Laravel application instance. It binds essential interfaces and prepares Laravel's service container.

Request is now sent to bootstrap/app.php

Two based on the request for routing:

- Web routing-> routes/web.php (hitting the URL via web browser)
- command -> routes/console.php

Service Provider

- the request is sent to Service Providers
- Implement core functionality e.g. Mailing, Sessions etc.

bootstrap/providers.php

- You can find a list of the user-defined or third-party service providers that your application is using in the bootstrap/providers.php file.

Service providers

- Service providers are responsible for bootstrapping all of the framework's various components, such as the database, queue, validation, and routing components, event management, session management.

Laravel comes with some default service providers:-

- App\Providers\AppServiceProvider.php
(in Laravel-11)
- App\Providers\AuthServiceProvider.php
- App\Providers\BroadcastServiceProvider.php
- App\Providers\EventServiceProvider.php
- App\Providers\RouteServiceProvider.php

Service Providers Loading (bootstrap/providers.php)

- Laravel loads all registered service providers to initialize core services like database, authentication, routing, etc.

Providers

- Config-> Appserviceprovider—to declare the global variables (custom stuff)

Routes (routes/web.php)

- Once the application has been bootstrapped and all service providers have been registered, the Request will be handed off to the **router** for dispatching.
- The router will dispatch the request to a **route or controller**, as well as run any route specific middleware.
- Laravel finds the appropriate route that matches the request URL.

Middleware (using construct method)

- **Middleware** provides a convenient mechanism for filtering or examining HTTP requests entering your application.
- For example, Laravel includes a middleware that verifies if the user of your application is authenticated.
- If the user is not authenticated, the middleware will redirect the user to the login screen. However, if the user is authenticated, the middleware will allow the request to proceed further into the application.

Middleware Execution

- Authenticate users.
- Log requests.
- Validate input data.

Example Middleware (app/Http/Middleware/CheckAge.php)

```
public function handle($request, Closure $next)
{
    if ($request->age < 18) {
        return response('Unauthorized', 403);
    }
    return $next($request);
}
```

Controller

- In Laravel, controllers are found in the `app/Http/Controllers` directory.
- They help organize request handling logic by grouping related functions into classes.

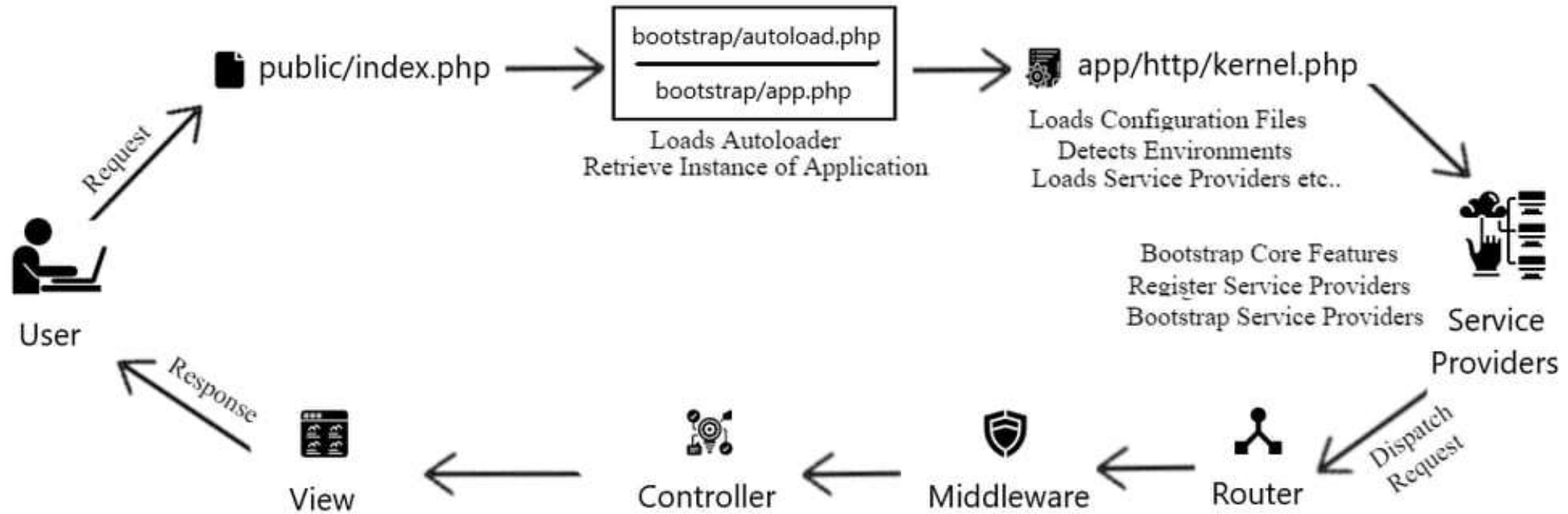
View

- If the route returns an HTML view, Laravel loads a Blade template.

(resources/views/home.blade.php)

```
<!DOCTYPE html>
<html>
<head>
  <title>
    {{ $title }}
  </title>
</head>
<body>
<h1>Hello, Laravel!</h1>
</body>
</html>
```

Laravel Request Life Cycle



By default laravel project runs on which PORT?

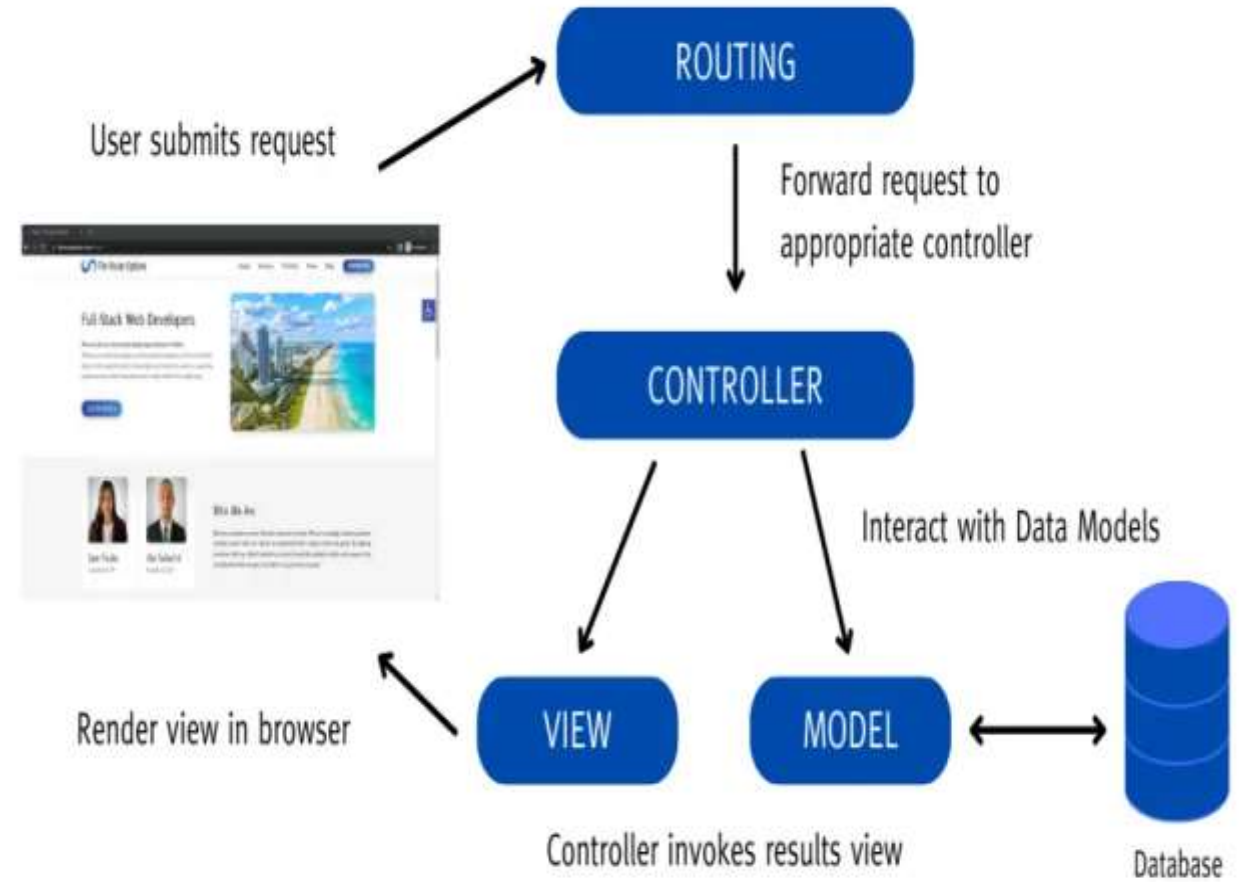
- 3000
- 5000
- 8000
- 7000

What's the command to check the current version of the Laravel framework?

- php artisan main --version
- php artisan --version status
- php artisan check --version
- php artisan --version

What is Routing?

The process of **directing incoming requests (usually based on URLs) to the appropriate code or handler** that will handle the request and generate a response.



Contd...

Routing is like a traffic director for web applications. When a user enters a URL in their browser or clicks a link, routing decides where that request should go within the application to generate the appropriate content or perform a specific action.

URL

URL stands for "Uniform Resource Locator." It's a reference or address used to access resources on the internet. In simpler terms, a URL is the **unique web address that you enter into a web browser to access a specific web page, file, or resource.**

By default one route is created by Laravel

```
use Illuminate\Support\Facades\Route;  
Route::get('/', function () {  
    return view('welcome');  
});
```

In the above case, Route is the class which defines the static method `get()`. The `get()` method contains the parameters `'/'` and the `function()` closure. The `'/'` defines the root directory, and `function()` defines the functionality of the `get()` method.

Paamayim Nekudotayim ::

- It is a token that allows access to static, constant, and overridden properties or methods of a class. When referencing these items from outside the class definition, use the name of the class.
- It's possible to reference the class using a variable. The variable's value can not be a keyword (e.g. self, parent and static).

Do you know?

Paamayim Nekudotayim would, at first, seem like a strange choice for naming a double-colon. However, while writing the Zend Engine 0.5 (which powers PHP 3), that's what the Zend team decided to call it. It actually does mean double-colon - in Hebrew!

Route::get('/')

- Route::get('/') defines a route for a GET request to the root URL (/) of your application.
- The root URL typically refers to the base of your web application, e.g., `http://yourdomain.com/`.

Closure Function:

- The second parameter of Route::get is a closure (anonymous function). This function will execute when the route is accessed.

Returning a View:

- Inside the closure, return `view('welcome')`; tells Laravel to render and return a Blade view named 'welcome'. Laravel assumes the view file is located at `resources/views/welcome.blade.php`.

What is the purpose of the web.php routes file in Laravel?

- **a) It defines routes for web application routes and middleware.**
- b) It defines routes specifically for API endpoints.
- c) It defines routes for console commands and tasks.
- d) It defines routes for unit testing

Command to get the route list in Laravel

- **php artisan route:list**

In Laravel, where are the routes primarily defined?

- a) In the app/routes.php file
- b) In the routes folder within the app directory
- c) In the .env configuration file
- **d) In the web.php or api.php files within the routes directory**

HTTP Methods

GET, POST, PUT, PATCH, and DELETE are the five most common HTTP methods for retrieving from and sending data to a server.

The GET method

The GET method is used to retrieve data from the server. **This is a read-only method, so it has no risk of mutating or corrupting the data.** For example, if we call the get method on our API, we'll get back a list of all to-dos.

The POST method

The POST method sends data to the server and creates a new resource. The resource it creates is subordinate to some other parent resource. When a new resource is POSTed to the parent, the API service will automatically associate the new resource by assigning it an ID (new resource URI). In short, **this method is used to create a new data entry**

The PUT method

The PUT method is most often **used to update an existing resource**. If you want to update a specific resource you can call the PUT method to that resource URI with the request body containing the complete new version of the resource you are trying to update.

The PATCH method

The PATCH method is very similar to the PUT method because it also modifies an existing resource. The difference is that for the PUT method, **the request body contains the complete new version, whereas for the PATCH method, the request body only needs to contain the specific changes to the resource**, specifically a set of instructions describing how that resource should be changed, and the API service will create a new version according to that instruction.

The DELETE method

The DELETE method is used to delete a resource specified by its URI.

View in Laravel

- Views provide a convenient way to place all of our HTML in separate files.
- Views separate your controller / application logic from your presentation logic and are stored in the resources/views directory. A simple view might look something like this:

```
<!-- View stored in resources/views/greeting.blade.php -->
<html>
  <body>
    <h1>Hello, {{ $name }}</h1>
  </body>
</html>
```

- Since this view is stored at resources/views/greeting.blade.php, we may return it using the global view helper like so:

```
Route::get('/', function () {
    return view('greeting', ['name' => 'James']);
});
```

Creating & Rendering Views

- You may create a view by placing a file with the `.blade.php` extension in your application's `resources/views` directory.
- The `.blade.php` extension informs the framework that the file contains a Blade template.
- Blade templates contain HTML as well as Blade directives that allow you to easily echo values, create "if" statements, iterate over data, and more.
- command to create the view is:

```
php artisan make:view user
```

 (here user is the name of the view).
- A view with name `user.blade.php` is created in `resources/views`.

Nested View Directories

- Views may also be nested within subdirectories of the resources/views directory.
- "Dot" notation may be used to reference nested views.
- For example, if your view is stored at resources/views/admin/profile.blade.php, you may return it from one of your application's routes / controllers like so:

```
return view('admin.profile', $data);
```

Passing Data To Views

- You may pass an array of data to views to make that data available to the view:

```
return view('greetings', ['name' => 'Victoria']);
```

- When passing information in this manner, the data should be an array with key / value pairs.
- After providing data to a view, you can then access each value within your view using the data's keys, such as `<?php echo $name; ?>`.

Contd...

- Inside Web.php

```
//Passing Data to View
```

```
Route::get('/test',function(){  
    $name="akash";  
    return view('test',['name'=>$name]);  
});
```

- create the view test:
in terminal: php artisan make:view test
- Inside test.blade.php

```
<h1>{{ $name }}</h1>
```

Passing Data To Views (contd.)

- As an alternative to passing a complete array of data to the view helper function, you may use the with method to add individual pieces of data to the view.
- The with method returns an instance of the view object so that you can continue chaining methods before returning the view:

return view('greeting')

->with('name', 'Victoria')

->with('occupation', 'Astronaut');

```
Route::get('/test',function(){  
    $name="akash";  
    $city="Jalandhar";  
    return view('test')  
        ->with('name',$name)  
        ->with('city',$city);  
});
```

One More Shortcut

- In web.php

```
Route::get('/test',function(){  
    $name="akash";  
    $city="Jalandhar";  
    return view('test')  
        ->withName($name)  
        ->withCity($city);  
});
```

- In test.blade.php

```
<h1>Name : {{$name}} <br> City: {{$city}}</h1>
```

Suppose empty string is passed in city

- Instead of printing empty string you can also use conditional expression in blade template file and print some default value.

```
<h1>Name : {{$name}} <br>  
    City: {!!empty($city)?$city:"Enter your City"!!}  
</h1>
```

Data passing methods

1. Using `with()` Method

```
return view('welcome')->with('name', 'Parveen');
```

In Blade (view file):

```
<h1>{{ $name }}</h1>
```

2. Using `compact()`

```
[$name = "Parveen", $age = 22];  
return view('welcome', compact('name', 'age'));
```

In Blade:

```
<h1>{{ $name }}</h1>  
<p>{{ $age }}</p>
```

`compact()` converts variables into an associative array:

```
['name' => $name, 'age' => $age]
```

Data passing methods

3. Passing Data as Array

```
return view('welcome', [ 'name' => 'Parveen', 'age' => 22]);
```

4. Passing Multiple Data Using **with()**

```
return view('welcome') ->with('name', 'Parveen') ->with('age', 22);
```

5. Passing Data from Route

```
Route::get('/test', function () {  
    $students = [  
        ['name' => 'Aman', 'roll' => 1],  
        ['name' => 'Riya', 'roll' => 2]  
    ];  
    return view('students', compact('students'));  
});
```

Data passing methods

7. Displaying Data in Blade (Loop)

```
@foreach($students as $student)
    <p>{{ $student['name'] }} - {{ $student['roll'] }}</p>
@endforeach
```

8. Passing Data to All Views (Global Data)

```
use Illuminate\Support\Facades\View;
```

```
View::share('appName', 'My Laravel App');
```

Now accessible in all views:

```
<h1>{{ $appName }}</h1>
```

9. Using **view()->with()** Helper

```
return view('welcome')->with([
    'name' => 'Parveen',
    'age' => 22
]);
```

What is the purpose of views in Laravel?

- A) To store database records.
- **B) To separate controller/application logic from presentation logic.**
- C) To define routes.
- D) To manage authentication.

Where are views typically stored in a Laravel application?

- A) In the public directory.
- B) In the app/views folder.
- **C) In the resources/views directory.**
- D) In the database folder.

Which file extension is used for Blade templates in Laravel views?

- A) .php
- B) .html
- **C) .blade.php**
- D) .view

In Blade templates, how can you access data that has been passed to a view?

- A) **By using the {{ data }} syntax.**
- B) By using echo data; syntax.
- C) By using \$data directly.
- D) By using the @data directive.

What method can you use to add individual pieces of data to a view in Laravel, allowing you to chain multiple data additions?

- A) addData()
- B) includeData()
- **C) with()**
- D) attachData()

How can you share data with all views rendered by your Laravel application?

- A) By storing it in a database.
- B) By using global variables.
- **C) By using the share method within a service provider's boot method.**
- D) By passing it to each view individually.

Sharing Data With All Views

- Occasionally, you may need to share data with all views that are rendered by your application.
- Typically, you should place calls to the share method within a service provider's boot method.
- You are free to add them to the App\Providers\AppServiceProvider class or generate a separate service provider to house them:

Sharing Data With All Views(contd.)

?php

```
namespace App\Providers;
```

```
class AppServiceProvider extends ServiceProvider
{
    public function boot()
    {
        view()::share('name', 'Pooja');
    }
}
```

on any view

```
<h2>User name is :{{$name}}<\h2>    // can be directly accessed
```

View Routes

- If your route only needs to return a view, you may use the `Route::view` method.
- This method provides a simple shortcut so that you do not have to define a full route or controller.
- The view method accepts a URI as its first argument and a view name as its second argument.

```
Route::view('/welcome', 'welcome');
```

Route Parameters

Required Parameters - Sometimes you will need to capture segments of the URI within your route. For example, you may need to capture a user's ID from the URL. You may do so by defining route parameters:

```
Route::get('/user/{id}', function ($id) {  
    return 'User '.$id;  
});
```

- You may define as many route parameters as required by your route.
- Route parameters are always encased within {} braces and should consist of alphabetic characters.
- Underscores (_) are also acceptable within route parameter names.

Contd...

Optional Parameters - Occasionally you may need to specify a route parameter that may not always be present in the URI. You may do so by placing a ? mark after the parameter name. Make sure to give the route's corresponding variable a default value:

```
Route::get('/user/{name?}', function ($name = null) {  
    return $name;  
});
```

```
Route::get('/user/{name?}', function ($name = 'John') {  
    return $name;  
});
```

Route Parameters Constraints

You may constrain the format of your route parameters using the family of 'where' methods on a route instance.

whereNumber Constraint:

The whereNumber constraint is used to ensure that a route parameter is a numeric value.

It is useful when you want to restrict a parameter to only accept numeric input.

```
Route::get('/user/{id}', function ($num) {  
    return 'User '.$num;  
})->whereNumber('id');
```

=> And -> are different

=> (Arrow or Fat Arrow)

- => is primarily used in association with key-value pairs, often seen in data structures like dictionaries, associative arrays, or objects in various programming languages.
- It is commonly used to assign values to keys or properties.

-> (Arrow or Member Access Operator)

- -> is used to access members (attributes or methods) of objects or structures in some programming languages, particularly in C-like languages such as C++ and PHP for object-oriented programming.
- It's used to access properties and methods of objects or structs.

whereAlpha

The whereAlpha constraint is used to ensure that a route parameter contains only alphabetic characters.

It restricts the parameter to accept only letters (A-Z and a-z).

```
Route::get('/user/{id}', function ($num) {  
    return 'User '.$num;  
})->whereAlpha('id');
```

whereAlphaNumeric

- The whereAlphaNumeric constraint is used to ensure that a route parameter contains only alphanumeric characters.
- It restricts the parameter to accept letters (A-Z and a-z) and numbers (0-9).

```
Route::get('/user/{id}', function ($num) {  
    return 'User '.$num;  
})->whereAlphaNumeric('id');
```

whereIn

- The whereIn constraint is used when you want to constrain a parameter's value to a specific set of values.
- It ensures that the parameter matches one of the specified values.

```
Route::get('/user/{id}', function ($num) {  
    return 'User '.$num;  
})->whereIn('id',['Attendance','Assignment']);
```

where

- The where method is a more general constraint that allows you to specify custom constraints using regular expressions or closures.
- It provides flexibility to define complex validation rules for a parameter.

```
Route::get('/user/{id}', function ($num) {  
    return 'User '.$num;  
})->where('id',"[0-9]+");
```

Handling Multiple Parameter Constraints

Suppose you have multiple parameters in URL, and you want to have different constraints for both, then one method for doing it is shown below.

```
Route::get('/user/{id}/{about}', function ($num) {  
    return 'User '.$num;  
})->where('id', "[0-9]+")->whereAlpha('about');
```

Route Parameters Global Constraint

- **Global Constraints** - In Laravel, you can define global route constraints by using the pattern method
- This allows you to apply constraints to all routes in your application. Global constraints can be useful for enforcing common validation rules on various route parameters throughout your application.

Route Parameters Global Constraint

- Here's how you can define a global route constraint for all routes:
- Open App/Providers/AppServiceProvider.php.

```
use Illuminate\Support\Facades\Route;  
public function boot()  
{  
    parent::boot();  
    Route::pattern('id', '[0-9]+');  
}
```

- In the above code, we're using the Route::pattern method to specify a regular expression pattern [0-9]+ for the id parameter. This pattern requires that any id parameter in your routes must consist of numeric characters.

What is Laravel routing primarily used for?

- A. Database management
- **B. Handling HTTP requests**
- C. Front-end development
- D. Server configuration

In Laravel, where are the routes typically defined?

- A. In the .env file
- B. In the config/routes.php file
- **C. In the routes/web.php and routes/api.php files**
- D. In the resources/views directory

Which of the following is used to pass parameters in a Laravel route?

- A. <param>
- B. [param]
- **C. {param}**
- D. (param)

Which artisan command is used to list all registered routes in a Laravel application?

- A. php artisan serve
- B. php artisan list
- C. php artisan routes
- **D. php artisan route:list**

What is the primary purpose of Laravel routing constraints?

- A. To define routes in the web.php file
- B. To restrict access to certain routes
- **C. To specify rules that route parameters must follow**
- D. To create dynamic routes

Which built-in constraint is used to ensure that a route parameter is numeric?

- **A. whereNumber**
- B. whereInteger
- C. whereNumeric
- D. whereDigit

Returning Response in Laravel

- The aim of the response is to provide the client with the resource it requested or inform the client that the action it requested has been carried out; or else, to inform the client that an error occurred in processing its request.

```
Route::get('/greet', function(){  
    return "Hello World";  
});
```

If we write "abc," it will open at this URL.

```
Route::get('/abc', function(){  
    return "Hello World";  
});
```

Creating Responses

- **Strings & Arrays** - All routes and controllers should return a response to be sent back to the user's browser.
- Laravel provides several different ways to return responses.
- The most basic response is returning a string from a route or controller. The framework will automatically convert the string into a full HTTP response:

```
Route::get('/', function () {  
    return 'Hello World';  
});
```

- In addition to returning strings from your routes and controllers, you may also return arrays.
- The framework will automatically convert the array into a JSON response:

```
Route::get('/', function () {  
    return [1, 2, 3];  
});
```

Creating Responses (contd.)

- **Response Objects –**

```
Route::get('/home', function () {  
    return response('Hello World', 200);  
});
```

Attaching Headers To Responses

- You may use the header method to add a series of headers to the response before sending it back to the user:

```
return response($content)  
    ->header('Content-Type', 'text/html')
```

View Responses

- If you need control over the response's status and headers but also need to return a view as the response's content, you should use the view method:

```
return response()  
    ->view('hello')  
    ->header('Content-Type', 'text/html');
```

JSON Responses

- The json method will automatically set the Content-Type header to application/json, as well as convert the given array to JSON using the json_encode PHP function.

```
return response()->json([  
    'name' => 'Abigail',  
    'state' => 'CA',  
]);
```

Do you know?

Douglas Crockford was the first to define and popularise the JSON format. In April 2001, Douglas Crockford and Chip Morningstar sent the first JSON message.

What is the primary goal of the response in Laravel?

- A) To confuse the client
- B) To provide irrelevant data
- **C) To provide the requested resource or inform about the outcome**
- D) To prevent the client from accessing resources

How can you return a string as a response in a Laravel route or controller?

- **A) return 'Hello World';**
- **B) return response('Hello World');**
- C) echo 'Hello World';
- D) Response::send('Hello World');

Which method should be used if you want to return view along with control over status and headers in Laravel?

- A) `return view('hello')`
- **B) `return response()->view('hello')`**
- C) `return respo('hello')->view()`
- D) `return view('hello')->re()`

What does the json method do in Laravel's response?

- A) Converts the response to plain text
- **B) Sets the response content to JSON format**
- C) Converts the response to XML
- D) Attaches JavaScript code to the response

Subrouting

```
Route::get('/home/page',function(){  
    return "<h1 style='color:red'> This is sub routing  
</h1>";  
});
```

Named Routes

```
Route::get('/name/lpu/about',function(){  
    return view("aboutus");  
});
```

Upon going on <http://127.0.0.1:8000/name/lpu/about>, it will open aboutus.blade.php

If I want to use the same route in multiple blade files

Laravel Named Routes

`http://localhost/page/about`

```
Route::get('/page/about', function () {  
    return 'About Page';  
});
```

first.blade.php

```
<a href='/page/about'>About</a>
```

second.blade.php

```
<a href='/page/about'>About</a>
```

third.blade.php

```
<a href='/page/about'>About</a>
```

Suppose I need to change the Route name

Laravel Named Routes

`http://localhost/page/about`

```
Route::get('/page/about-us', function () {  
    return 'About Page';  
});
```

`first.blade.php`

```
<a href='/page/about'>About</a>
```

`second.blade.php`

```
<a href='/page/about'>About</a>
```

`third.blade.php`

```
<a href='/page/about'>About</a>
```

Add name('about'); in Routes

Laravel Named Routes

<http://localhost/page/about>

```
Route::get('/page/about-us', function () {  
    return 'About Page';  
})->name('about');
```

first.blade.php

```
<a href='/page/about'>About</a>
```

second.blade.php

```
<a href='/page/about'>About</a>
```

third.blade.php

```
<a href='/page/about'>About</a>
```

Add route function in blade template



Laravel Named Routes

<http://localhost/page/about>

```
Route::get('/page/about-us', function () {  
    return 'About Page';  
})->name('about');
```

first.blade.php

```
<a href='{{ route('about') }}'>About</a>
```

second.blade.php

```
<a href='{{ route('about') }}'>About</a>
```

third.blade.php

```
<a href='{{ route('about') }}'>About</a>
```

Giving the name to route with name('about');
Use route ('about') in Blade template

Benefits of named route

- No need to change the <a> tag
- Instead of using parent and sub-route naming (which is very large), we use named routes.

Anchor Tag

```
<h1> About Us </h1>
```

```
<h1> {{ $name }} </h1>
```

```
<a href="/" > Welcome Page </a>
```

The Route for / must be declared in web.php

```
Route::get('/', function () { // URL to open  
    return view('welcome');  
});
```

Output-On Clicking Welcome Page,
<http://127.0.0.1:8000> will get open



Redirection

Redirection in Laravel is used to send users from one URL/route to another. It's very common after form submission, login, logout, or when access is restricted.

- The simplest method is to use the global redirect helper:

```
Route::get('/dashboard', function () {  
    return redirect('home/dashboard');  
});
```

- **Redirect to previous page**

```
return redirect()->back();
```

- **Redirect to Named Routes**

```
Route::get('/dashboard', function () {  
    return "Dashboard";  
})->name('dashboard');
```

Redirect

```
return redirect()->route('dashboard');
```

- **Redirect to Controller Action**

```
return redirect()->action([UserController::class, 'index']);
```

- **Redirect with Data (Flash Session)**

Pass success message

```
return redirect('/home')->with('success', 'Login successful!');
```

Access in Blade

```
@if(session('success'))  
    <p>{{ session('success') }}</p>  
@endif
```

- **Redirect with Input Data**

```
return redirect()->back()->withInput();
```

Keeps old form values

In Blade

```
<input type="text" name="name" value="{{ old('name') }}">
```

- **Redirect with Errors**

```
return redirect()->back()->withErrors(['name' => 'Name is required']);
```

Used in validation

- **Redirect with HTTP Status Code**

```
return redirect('/home', 301);
```

- 302 → Temporary (default)
- 301 → Permanent

- **Redirect to External URL**

```
return redirect()->away('https://google.com');
```

Bypasses Laravel URL validation

- **Redirect in Routes**

```
Route::get('/old-page', function () {  
    return redirect('/new-page');  
});
```

- **Redirect using Route Shortcut**

```
Route::redirect('/old', '/new', 301);
```

- **Redirect to fallback**

```
Route::fallback('/route', function () {  
    return "page not existing;  
});
```

// Here for the route, pass the route value that is not existing in the project, and then instead of 404/not found, it will display this return page not existing.